

Basic React Router Interview Questions & Answers

1. What is `react-router-dom` and why do we use it?

`react-router-dom` is a **library** for handling **routing** in React applications. It allows you to create **single-page applications (SPA)** with multiple views without reloading the page. This provides a smooth user experience similar to multi-page apps but without full-page refreshes.

2. How is client-side routing different from traditional server-side routing?

- **Client-side routing** (React Router) updates only parts of the UI without reloading the page. Navigation happens instantly.
- **Server-side routing** (traditional websites) makes a request to the server every time a user navigates, causing full-page reloads.

3. What is `<BrowserRouter>` and why do we wrap our main component inside it?

`<BrowserRouter>` is the **main component** that enables routing in a React app. It **keeps track of the URL changes** and renders the correct component.

We wrap our main component inside it to allow all child components to use routing features.

Example:

```
import { BrowserRouter } from "react-router-dom";
import App from "./App";
```

```
const Root = () => (
  <BrowserRouter>
    <App />
  </BrowserRouter>
)
```

```
);
```

```
export default Root;
```

So, What Does `<BrowserRouter>` Do?

- ☒ Listens to URL changes in the browser.
- ☒ Maps URLs to Components without full page reload.
- ☒ Enables navigation using `<Link>` or `<Route>`.

4. What are `<Routes>` and `<Route>` in React Router?

- `<Routes>` acts as a container for defining different `<Route>` components.
- `<Route>` maps a **path** to a **component** that should be rendered.

Example:

```
import { Routes, Route } from "react-router-dom";  
import Home from "../Home";  
import About from "../About";
```

```
function App() {  
  return (  
    <Routes>
```

```

    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
  </Routes>
);
}

export default App;

```

Here, when the URL is `/`, the `Home` component is displayed. When it's `/about`, the `About` component is shown.

5. How do you navigate between pages without reloading using `react-router-dom`?

Instead of `<a>` tags (which cause full-page reloads), we use `<Link>` from `react-router-dom`.

Example:

```

import { Link } from "react-router-dom";

function NavBar() {
  return (
    <nav>
      <Link to="/">Home</Link>
      <Link to="/about">About</Link>
    </nav>
  );
}

```

Clicking a `<Link>` updates the URL **without refreshing the page**.

6. What is the difference between `<Link>` and `<a>` tag in React Router?

- `<a>` tag reloads the entire page when clicked.

- `<Link>` updates the URL and changes the component **without reloading** the page.

Example:

```
<Link to="/about">Go to About</Link> // No page refresh  
<a href="/about">Go to About</a> // Full page reload
```

For **better performance**, always use `<Link>` in React apps.

GIT Commands

Steps to push project into Repo

Git init

Git remote -v

Git remote add origin url

Git add .

Git commit -m "message"

Git branch

Git branch -M main

Git push origin main

After changes to the file

Git status

Git add .

Git commit -m "message"

Git push origin main

git checkout -b feature1

git branch

git checkout main

git branch

git checkout feature1

git branch

git add .

git commit -m "Added feature 1"

git push origin feature1

HOSTING REACT PROJECT

1. Build the Production Version

Before hosting, you must create an optimized version of your app.

If you used **Create React App**:

```
npm run build
```

If you used **Vite**:

```
npm run build
```

This creates a build (or dist for Vite) folder — the one you'll upload to hosting.

Choose a Hosting Option

OPTION 1: GitHub Pages (Free & Easy)

Perfect for simple projects and portfolios.

Steps:

1. Install GitHub Pages package:

```
npm install gh-pages --save-dev
```

2. In your package.json, add:

```
"homepage": "https://yourusername.github.io/your-repo-name",  
"scripts": {  
  "predeploy": "npm run build",  
  "deploy": "gh-pages -d build"  
}
```

Example:

```
{  
  "name": "taskbuddy",  
  "private": true,  
  "version": "0.0.0",  
  "type": "module",  
  "homepage": "https://<your-username>.github.io/taskbuddy",  
  "scripts": {  
    "dev": "vite",  
    "build": "vite build",  
    "preview": "vite preview",  
    "predeploy": "npm run build",  
    "deploy": "gh-pages -d dist"  
  },  
  "dependencies": {  
    "react": "^19.0.0",  
    "react-dom": "^19.0.0"  
  },  
}
```

```
"devDependencies": {  
  "@vitejs/plugin-react": "^4.3.4",  
  "vite": "^6.2.0",  
  "gh-pages": "^6.3.0"  
}
```

In your project root, open (or create) vite.config.js and add this:

```
import { defineConfig } from 'vite'  
import react from '@vitejs/plugin-react'  
  
export default defineConfig({  
  plugins: [react()],  
  base: '/taskbuddy/', // 👉 must match your repo name  
})
```

3. Initialize Git and push your app:

```
git init  
git add .  
git commit -m "Initial commit"  
git remote add origin https://github.com/yourusername/your-repo-name.git  
git push -u origin main
```


Build your app

npm run build

Deploy to GitHub Pages

npm run deploy

That's it!

After a few seconds, your app will be live at:

👉 <https://your-username.github.io/taskbuddy>

OPTION 2: Host on Netlify (Recommended for Vite)

No config changes needed!

Steps:

1. Push your code to GitHub.
2. Go to <https://app.netlify.com>
3. Click “**Add new site**” → “**Import from Git**”
4. Choose your repo (taskbuddy)
5. In build settings:
 - a. **Build command:** npm run build
 - b. **Publish directory:** dist
6. Click **Deploy**

✓ Netlify gives you a free live URL like:
<https://taskbuddy.netlify.app>

OPTION 3: Host on Vercel (Super smooth for React/Vite)

Steps:

1. Push your project to GitHub.
2. Go to <https://vercel.com>
3. Click **“Add New Project”**
4. Import your repo (taskbuddy)
5. Vercel detects Vite automatically.
6. Click **Deploy**

✓ Live link: <https://taskbuddy.vercel.app>